How the Internet Delivers a Web Page

Case Study: www.youtube.com

Course: PCCST501 – Computer Networks | Module 1 Assignment

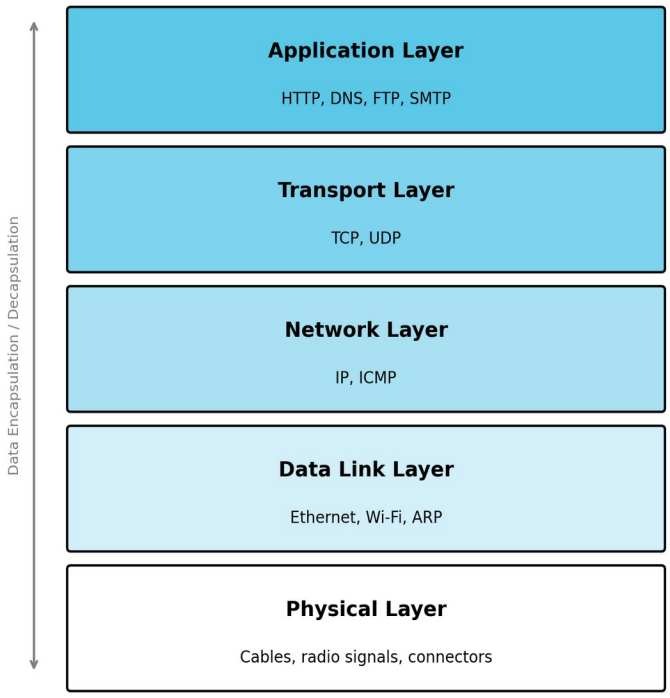
Introduction

Every time a user types a URL such as https://www.youtube.com into a web browser and presses Enter, a complex sequence of events unfolds within a fraction of a second. This report traces that entire journey step by step — from resolving the domain name to establishing a reliable connection, exchanging data using HTTP, and finally rendering the page on screen. Each stage is mapped to the layer of the TCP/IP protocol stack responsible for it, so that the overall picture of how the Internet delivers a web page becomes clear.

1. The TCP/IP Protocol Stack

The TCP/IP model organizes network communication into four layers, each handling a specific part of the job of moving data from one machine to another. The diagram below shows the stack from the application layer (closest to the user) down to the network access layer (closest to the physical medium).

TCP/IP Protocol Stack (5-Layer Model)



Role of each layer:

Layer	Role	Example Protocols
Application Layer	Provides network services directly to end-user applications; formats and interprets data such as web pages, emails, and file transfers.	HTTP, HTTPS, DNS, FTP, SMTP
Transport Layer	Provides end-to-end (process-to-process)	TCP, UDP

	communication, segmentation, flow control, and reliability (TCP) or fast best-effort delivery (UDP).	
Network Layer	Handles logical (IP) addressing and routing of packets across different networks so they reach the correct destination host.	IP, ICMP
Data Link Layer	Provides node-to-node delivery within the same network; organizes bits into frames, adds MAC addressing, and performs frame-level error detection.	Ethernet, Wi-Fi (802.11), ARP
Physical Layer	Transmits raw bits as electrical, optical, or radio signals over the actual physical transmission medium between devices.	Cables, connectors, radio signals, voltages

When YouTube's home page is requested, the browser's HTTP request (Application layer) is handed to TCP (Transport layer), which breaks it into segments and adds sequence numbers; TCP then hands segments to IP (Network layer), which adds source/destination addresses and determines the route; the Data Link layer then wraps each packet into a frame with the appropriate MAC addressing; and finally, the Physical layer transmits the resulting bits as electrical, optical, or radio signals over Wi-Fi or Ethernet. The response from YouTube's server travels back up through the same five layers in reverse — a process called encapsulation on the way down and decapsulation on the way up.

2. Role of DNS in Resolving the Domain Name

Computers communicate using IP addresses, not human-friendly names, so the Domain Name System (DNS) acts as the Internet's phone book, translating www.youtube.com into an IP address such as 142.250.183.14. The resolution process typically involves several steps:

- The browser first checks its own cache, then the operating system's cache, for a previously resolved IP address.
- If not found, the request goes to a DNS resolver (usually provided by the ISP), which queries a root DNS server to find the authoritative server for the '.com' top-level domain (TLD).
- The TLD server directs the resolver to YouTube's/Google's authoritative name servers.
- The authoritative server returns the specific IP address mapped to www.youtube.com.
- The resolver caches this result and returns it to the browser, which can now open a connection to that IP address.

This entire lookup usually takes only a few milliseconds and is largely invisible to the user, but it is an essential precondition for every other step that follows.

3. HTTP: Requesting and Receiving the Web Page

Once the browser has YouTube's IP address and a TCP connection is established, it uses the HyperText Transfer Protocol (HTTP, or more commonly HTTPS for encrypted traffic) to actually fetch the content. The browser sends an HTTP GET request containing the requested path (for example, '/'), the HTTP version, and header fields such as the browser type, accepted content formats, and cookies. YouTube's web server processes this request and sends back an HTTP response, which includes a status code (such as 200 OK), response headers, and the body — typically an HTML document. The browser then parses this HTML and issues further HTTP requests automatically for embedded resources such as CSS stylesheets, JavaScript files, images, and video thumbnails, each following the same request–response pattern, until the full page is assembled.

4. How TCP Provides Reliable Communication for HTTP

HTTP itself does not guarantee delivery of data; that reliability is provided by TCP (Transmission Control Protocol) operating beneath it. TCP achieves reliable, ordered, and error-checked delivery through several mechanisms:

- Three-way handshake: before any data is exchanged, the client and server synchronize using SYN, SYN-ACK, and ACK segments to establish a connection.
- Sequence numbers: every byte of data is numbered, allowing the receiver to reassemble segments in the correct order even if they arrive out of sequence.
- Acknowledgements and retransmission: the receiver acknowledges received segments; if an acknowledgement is not received in time, TCP retransmits the lost segment.
- Flow control and congestion control: TCP adjusts the rate of data transmission based on the receiver's buffer capacity and network congestion, preventing data loss and overload.
- Error checking: checksums detect corrupted segments so they can be discarded and retransmitted.

Because of these guarantees, the HTML, CSS, JavaScript, and video data that make up the YouTube page arrive completely and in the correct order, even though the underlying IP layer only offers best-effort delivery.

5. Client–Server vs Peer-to-Peer Architecture

YouTube follows a Client–Server architecture rather than a Peer-to-Peer (P2P) model. In this architecture, YouTube operates large, centralized (though geographically distributed) server infrastructure and content delivery networks (CDNs) that store and serve video content and web pages, while each user's browser acts purely as a client that requests and consumes this content.

This classification is justified by several observations: end users never communicate directly with one another to exchange video data; all requests are routed through YouTube's servers, which have far greater processing power and storage than any individual client; the server maintains authoritative content, user accounts, and video catalogs; and access control, authentication, and content delivery are centrally managed. This is in direct contrast to P2P systems (such as BitTorrent), where every node can act as both client and server and content is exchanged directly between peers without a central authority controlling the exchange.

6. HTTP vs FTP – Key Differences

Basis	HTTP	FTP
Purpose	Used to request and deliver web pages and resources between browser and server.	Used specifically to transfer files between a client and a server.
Connections used	Uses a single connection to carry both requests and responses (control and data together).	Uses two separate connections: one for control commands (port 21) and one for data transfer (port 20).
State	Stateless by default; each request is independent (state is added via cookies/sessions).	Maintains a stateful session for the duration of the file-transfer connection.
Default port	Port 80 (HTTP) / Port 443 (HTTPS)	Port 21 (control) and Port 20 (data)
Typical use case	Browsing websites, loading web applications, APIs.	Uploading/downloading large files to/from a remote server.

7. Cookies and User Experience

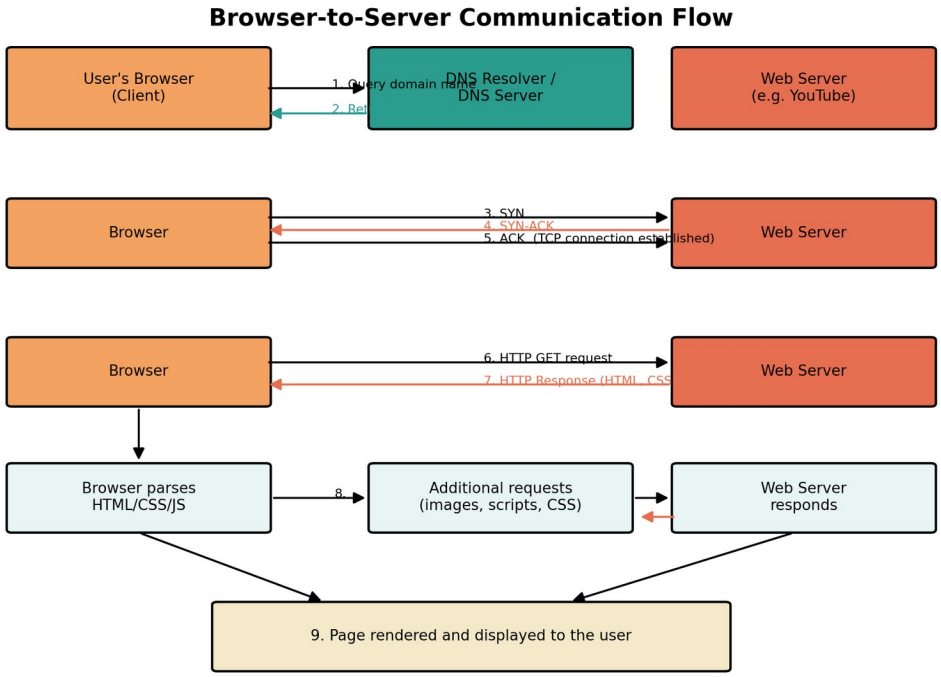
Cookies are small pieces of data that a web server asks the browser to store and send back with subsequent requests to the same site. Because HTTP is inherently stateless — treating every request as independent — cookies

give websites a way to 'remember' information about a user across multiple requests or visits, greatly improving the browsing experience.

Real-world example: On YouTube, cookies are used to keep a user signed in across sessions, remember video playback preferences (such as quality settings or autoplay), retain items in a 'Watch Later' list between visits, and personalize video recommendations based on viewing history. Without cookies (or similar session-tracking mechanisms), a user would have to log in and reset their preferences every single time they loaded a new page, since the server would have no memory of who they were from one request to the next.

8. Flow Diagram: Browser-to-Server Communication

The diagram below summarizes the complete sequence of communication described in the sections above, from the moment the browser resolves the domain name to the moment the page is rendered on screen.



9. Conclusion

Application-layer protocols such as DNS and HTTP form the essential bridge between human intention and machine-to-machine communication. DNS spares users from having to remember numeric IP addresses, while HTTP defines a consistent, universal language through which browsers and servers exchange web content. These protocols, working together with the reliability guarantees of TCP and the addressing and routing functions of IP, allow something as simple as typing 'youtube.com' to trigger a coordinated exchange across multiple networks, servers, and devices within milliseconds. Without application-layer protocols, every website would need its own custom communication method, making the web fragmented and unusable at scale. Their standardization is precisely what allows billions of users to seamlessly access services like YouTube, Gmail, and Instagram every day, making application-layer protocols one of the most important foundations of everyday Internet usage.